Problem/Challenge:

1. Implement infinite scroll product listing page with json fetched from the below url:
   GET "https://www.ounass.ae/api/v2/women/clothing"

   Some important fields from response: name, designerCategoryName, price, thumbnail, noFilterUrl

   Pagination: Pagination related fields are under plp -> pagination object

2. Replicate the capabilities of product detail page with this example json:
   GET "https://www.ounass.ae/api/v2/{slug}.html"
   - **slug** = represents the slug of the product received from the product list response

   Base url for images: Product Detail Images Base URL

   Some important fields from response: name, designerCategoryName, descriptionText, price, media, amberPoints

3. Document the solution and how to run the project.
   Example questions that may be helpful about documentation:
   - Why have you selected that design pattern?
   - Which software design principles have you followed in your project?
   - Any assumptions/comments/notes about any particular decision?
   - What would you change if you had more time?
   - What to do to make the app production ready?
   - Does your project require any particular tool to be able to run?

Please keep in mind that **some important fields from response** information is provided just for the sake of simplicity. Based on your implementation and UI, you may need more fields or you can skip some of them.

Ounass app, available at the app store, can be used as a reference for UI and functionality: Ounass iOS

Implementation:

Implementation should include:

- Product listing with infinite scroll / lazy loading and pull to refresh capabilities
- Required network calls
- Navigation to product detail for selected product from product listing screen (Every item should route to its own product detail screen, above product detail url is just an example)
- Size and color selection in product detail screen
- The state of the **Add to Bag** button, based on size and color selection (Actual functionality and network call for Add to Bag **is not** required)

Example Scenarios:

- If you select a color from the product detail screen, size selection should be limited with available sizes for the selected color.
- If a product does not have more than one color option available, color selection should not be activated, same applies to size selection also. (See products with only size options available, only color options available and none of them available from the actual app for the acceptable functional behavior)
- On size or color selection, product ID, description and images should be updated.

Evaluation Criteria:

- Implementation of the logic and UI from mobile development perspective
- Ability to easily add new options other than color and size
- Code organization and design pattern that is easily adaptable for multi brand development, easy to maintain and test (Strictly, **NO** Apple MVC)
- Consistency on code convention and indentation
- Swift and Xcode versions and deployment target (Swift 5.0 or higher, latest version of stable Xcode, deployment target of 13.0 and higher)
- Portrait/Landscape orientations and iPad compatibility (Different UI for landscape orientation and iPad **is not** required)
- **Swifty** mindset (Ability to gain advantage from swift's powerful features while overcoming the pitfalls)
- Library usage (No more and less than required)